TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

UTN
UNIVERSIDAD TECNOLÓGICA NACIONAL

Anexo de Evidencias

Trabajo Práctico Integrador

Metodología de Sistemas II

Integrantes:

- Grela, Ariel
- Higa, Matías

Comisión N° 1

M25 C4-06

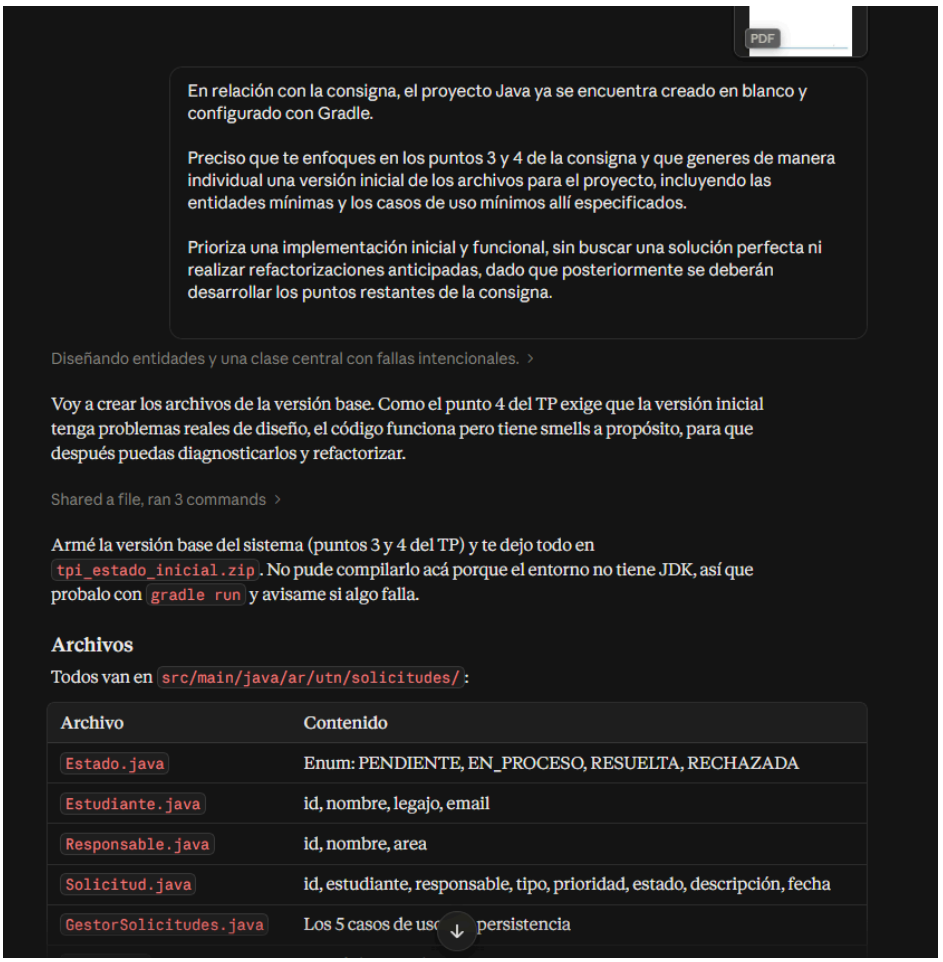
Tabla de Contenidos

Tabla de Contenidos **2**
1. Capturas de Pantalla 📷 **4**
CAP-01 4
CAP-02 4
CAP-03 5
CAP-04 5
CAP-05 6
CAP-06 7
CAP-07 8
CAP-08 9
CAP-09 9
CAP-10 10
CAP-11 10
CAP-12 11
CAP-15 12
CAP-16 12
CAP-17 13
CAP-18 13
CAP-19 13
CAP-20 14
CAP-21 14
CAP-22 14
CAP-23 14
CAP-24 15
CAP-25 15
CAP-26 16
CAP-27 16
CAP-28 17
CAP-29 17
CAP-30 17
CAP-31 18
CAP-32 18
CAP-33 18
CAP-34 19
CAP-35 19
CAP-36 20
CAP-37 20
CAP-38 20
CAP-39 21
CAP-40 21

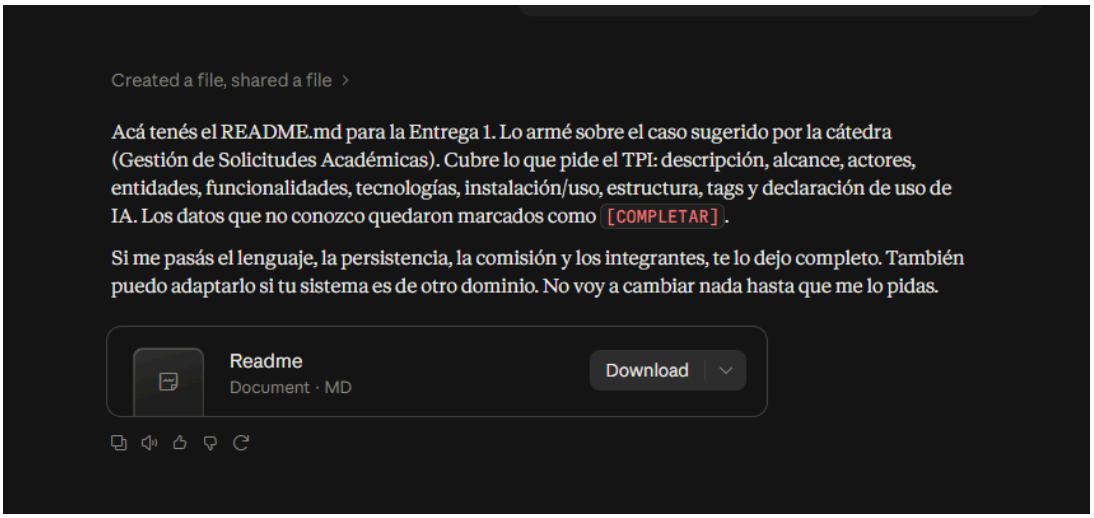
CAP-41 21
CAP-42 21
CAP-43 22
CAP-44 22
CAP-45 23
CAP-46 24
CAP-47 25
CAP-48 25
CAP-49 25
CAP-50 25
CAP-51 26
CAP-52 26
CAP-53 26
CAP-54 27
CAP-55 27
CAP-56 28
2. Uso de IA 🤖 29
2.1. Declaración general 29
2.2 Escala de grado de intervención 29
2.3 Tabla resumen de interacciones 29
2.4 Registro por interacción 30
IA-01 30
IA-02 31
IA-03 31
IA-04 32
IA-05 32
3. Bibliografía y recursos consultados 34

1. Capturas de Pantalla 📷

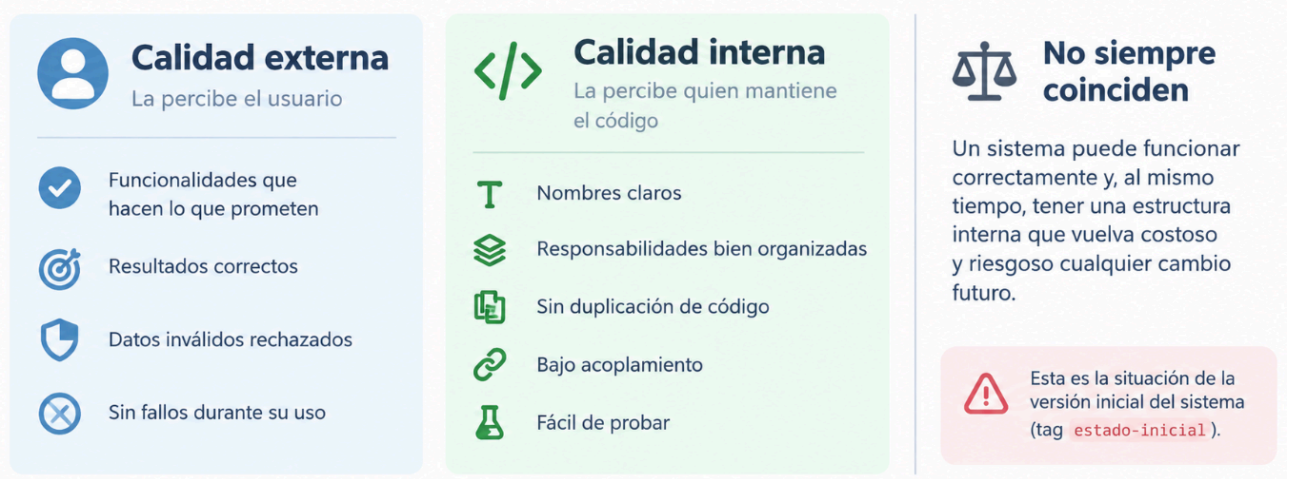
CAP-01



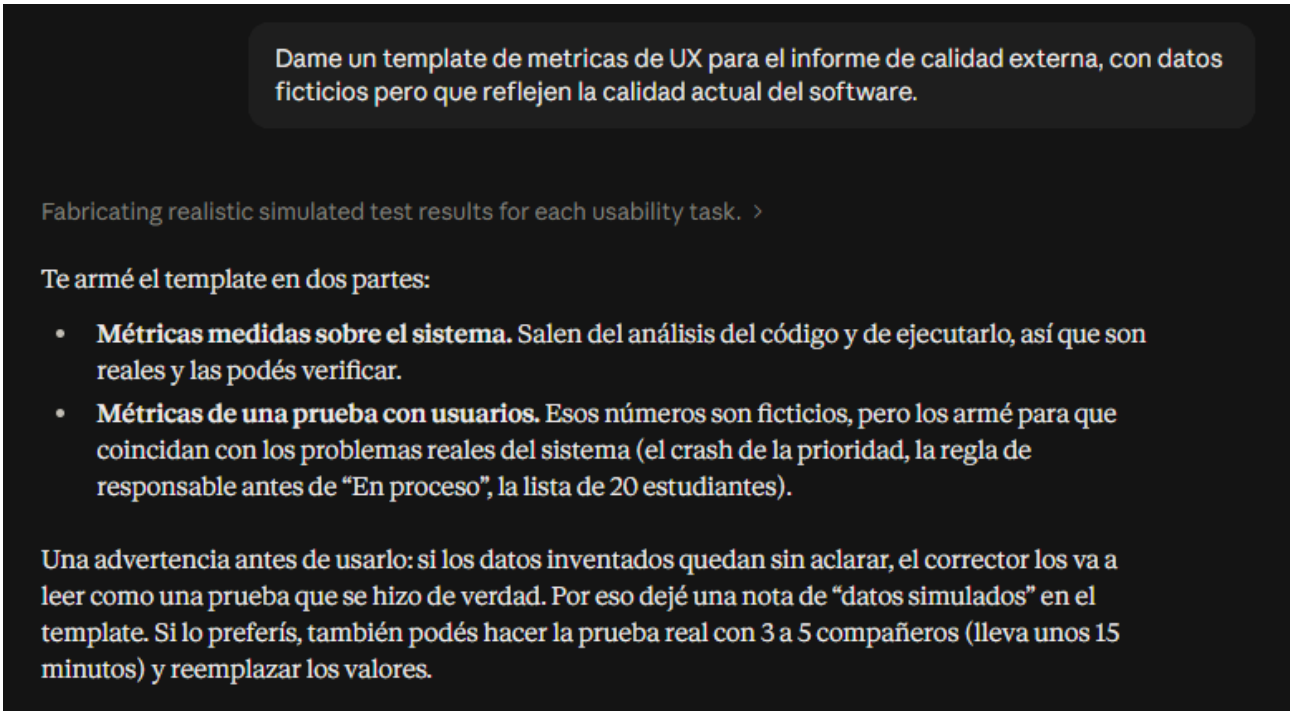
CAP-02



CAP-03


Calidad externa
La percibe el usuario
Funcionalidades que hacen lo que prometen
Resultados correctos
Datos inválidos rechazados
Sin fallos durante su uso
Calidad interna
La percibe quien mantiene el código
Nombres claros
Responsabilidades bien organizadas
Sin duplicación de código
Bajo acoplamiento
Fácil de probar
No siempre coinciden
Un sistema puede funcionar correctamente y, al mismo tiempo, tener una estructura interna que vuelva costoso y riesgoso cualquier cambio futuro.
Esta es la situación de la versión inicial del sistema (tag estado-inicial).

CAP-04


Dame un template de metricas de UX para el informe de calidad externa, con datos ficticios pero que reflejen la calidad actual del software.
Fabricating realistic simulated test results for each usability task. ›
Te armé el template en dos partes:
• Métricas medidas sobre el sistema. Salen del análisis del código y de ejecutarlo, así que son reales y las podés verificar.
• Métricas de una prueba con usuarios. Esos números son ficticios, pero los armé para que coincidan con los problemas reales del sistema (el crash de la prioridad, la regla de responsable antes de "En proceso", la lista de 20 estudiantes).
Una advertencia antes de usarlo: si los datos inventados quedan sin aclarar, el corrector los va a leer como una prueba que se hizo de verdad. Por eso dejé una nota de "datos simulados" en el template. Si lo preferís, también podés hacer la prueba real con 3 a 5 compañeros (lleva unos 15 minutos) y reemplazar los valores.

CAP-05

```
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 18
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 3
Descripcion: Nota de parcial 1 de Gestión todavía no cargada en campus
Prioridad (1=alta, 2=media, 3=baja): 1
Solicitud registrada con id 4
```

CAP-06

```
Opcion: 1
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 11
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 4
Descripcion: hola
Prioridad (1=alta, 2=media, 3=baja): 1
Error: descripcion muy corta
```

CAP-07

```
Estudiante:
1. L1001 - Agustin Di Maria (agustin87@gmail.com)
2. L1002 - Lucia Kempes (lucia.kempes@outlook.com)
3. L1003 - Tomas Maradona (tomas.mdz@hotmail.com)
4. L1004 - Valentina Passarella (vale.passarella@yahoo.com)
5. L1005 - Nicolas Fillol (nfillol23@gmail.com)
6. L1006 - Camila Batistuta (camila.batistuta@outlook.com)
7. L1007 - Franco Houseman (fr.houseman@gmail.com)
8. L1008 - Martina Ardiles (martina.ardiles@hotmail.com)
9. L1009 - Santiago Bertoni (sbert0ni@yahoo.com)
10. L1010 - Julieta Valdano (julieta.valdano@gmail.com)
11. L1011 - Mateo Ruggeri (mateo.ruggeri@outlook.com)
12. L1012 - Florencia Gallego (flor.gallego@hotmail.com)
13. L1013 - Benjamin Tarantini (benja.tarantini@gmail.com)
14. L1014 - Sofia Houseman (sofia.h77@yahoo.com)
15. L1015 - Facundo Rattin (facundo.rattin@outlook.com)
16. L1016 - Micaela Sivori (mica.sivori@gmail.com)
17. L1017 - Ignacio Labruna (ignacio.labruna@hotmail.com)
18. L1018 - Paula Ayala (paula.ayala@yahoo.com)
19. L1019 - Agustin Caniggia (agus.caniggia@gmail.com)
20. L1020 - Julian Brown (julian.brown@outlook.com)
Opcion: 5
Tipo:
1. CONSTANCIA
2. EQUIVALENCIA
3. RECLAMO
4. OTRO
Opcion: 2
Descripcion: Analisis Matematico -> Matematica II
Prioridad (1=alta, 2=media, 3=baja): 19
Error: prioridad invalida
```

CAP-08

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | PENDIENTE | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
La solicitud #1 no tiene responsable asignado. Solo puede rechazarse; para otros estados asignelo desde la opcion 3.
Nuevo estado:
1. RECHAZADA
Opcion:
```

CAP-09

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | RECHAZADA | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | EN_PROCESO | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 2
Nuevo estado:
1. EN_PROCESO
2. RESUELTA
3. RECHAZADA
Opcion: 2
Estado actualizado a RESUELTA
```

CAP-10

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 2
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | RECHAZADA | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | RESUELTA | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 1
Error: la solicitud esta cerrada
```

CAP-11

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 3
--- Solicitudes ---
#1 | Agustin Di Maria | CONSTANCIA | Prioridad 2 | RECHAZADA | SIN ASIGNAR
#2 | Lucia Kempes | RECLAMO | Prioridad 1 | RESUELTA | Maria Ortega S◆nchez (Alumnado)
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad 3 | PENDIENTE | SIN ASIGNAR
#4 | Paula Ayala | RECLAMO | Prioridad 1 | PENDIENTE | SIN ASIGNAR
#5 | Julieta Valdano | EQUIVALENCIA | Prioridad 1 | PENDIENTE | SIN ASIGNAR
Id solicitud: 3
Responsable:
1. Maria Ortega S◆nchez (Alumnado)
2. Jorge Tchami (Alumnado)
3. Laura Gorosito (Secretaria Academica)
Opcion: 2
Responsable asignado: Jorge Tchami
```

CAP-12

```
####################### GESTION DE SOLICITUDES ACADEMICAS #######################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 4
Buscar por:
1. Estado
2. Tipo
3. Legajo
4. Prioridad
Opcion: 1
Estado:
1. PENDIENTE
2. EN_PROCESO
3. RESUELTA
4. RECHAZADA
Opcion: 2
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | EN_PROCESO
Resultados: 1
```

CAP-13

```
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 4
Buscar por:
1. Estado
2. Tipo
3. Legajo
4. Prioridad
Opcion: 3
Estudiante:
1. L1001 - Agustin Di Maria
2. L1002 - Lucia Kempes
3. L1003 - Tomas Maradona
4. L1004 - Valentina Passarella
5. L1005 - Nicolas Fillol
6. L1006 - Camila Batistuta
7. L1007 - Franco Houseman
8. L1008 - Martina Ardiles
9. L1009 - Santiago Bertoni
10. L1010 - Julieta Valdano
11. L1011 - Mateo Ruggeri
12. L1012 - Florencia Gallego
13. L1013 - Benjamin Tarantini
14. L1014 - Sofia Houseman
15. L1015 - Facundo Rattin
16. L1016 - Micaela Sivori
17. L1017 - Ignacio Labruna
18. L1018 - Paula Ayala
19. L1019 - Agustin Caniggia
20. L1020 - Julian Brown
Opcion: 1
#1 | Agustin Di Maria | CONSTANCIA | Prioridad: 2 | RECHAZADA
#3 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | EN_PROCESO
Resultados: 2
```

CAP-14

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 5
===== REPORTE DE SOLICITUDES =====
#1 | 2026-10-22 | Agustin Di Maria | CONSTANCIA | Prioridad: 2 | RECHAZADA | SIN ASIGNAR
#2 | 2026-10-22 | Lucia Kempes | RECLAMO | Prioridad: 1 | RESUELTA | Maria Ortega S◆nchez (Alumnado)
#3 | 2026-10-22 | Agustin Di Maria | EQUIVALENCIA | Prioridad: 3 | EN_PROCESO | Jorge Tchami (Alumnado)
#4 | 2026-10-22 | Paula Ayala | RECLAMO | Prioridad: 1 | PENDIENTE | SIN ASIGNAR
#5 | 2026-10-22 | Julieta Valdano | EQUIVALENCIA | Prioridad: 1 | PENDIENTE | SIN ASIGNAR
---------------------------------
Total: 5
Pendientes: 2 | En proceso: 1 | Resueltas: 1 | Rechazadas: 1
Alta: 3 | Media: 1 | Baja: 1
```

CAP-15

```
######################## GESTION DE SOLICITUDES ACADEMICAS ########################
1. Registrar solicitud
2. Cambiar estado
3. Asignar responsable
4. Buscar / filtrar
5. Reporte
0. Salir
Opcion: 666
Opcion invalida
```

CAP-16

```
Execution failed for task ':tupad.ms2.Main.main()'.
> Process 'command 'C:\Users\matut\.jdks\ms-21.0.12.1\bin\java.exe'' finished with non-zero exit value 1

* Try:
> Run with --stacktrace option to get the stack trace.
> Run with --info or --debug option to get more log output.
> Run with --scan to get full insights from a Build Scan (powered by Develocity).
> Get more help at https://help.gradle.org.
BUILD FAILED in 17m 39s
2 actionable tasks: 1 executed, 1 up-to-date
```

CAP-17

```
                String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                int t = elegir(sc, titulo: "Tipo:", tipos);
                if (t == -1) {
                    System.out.println("Registro cancelado.");
                } else {
                    System.out.print("Descripcion: ");
                    String d = sc.nextLine();
                    System.out.print("Prioridad (1=alta, 2=media, 3=baja): ");
                    int p = Integer.parseInt(sc.nextLine());
                    g.registrarSolicitud(est.legajo, tipos[t], d, p);
                }
            }
        }
```

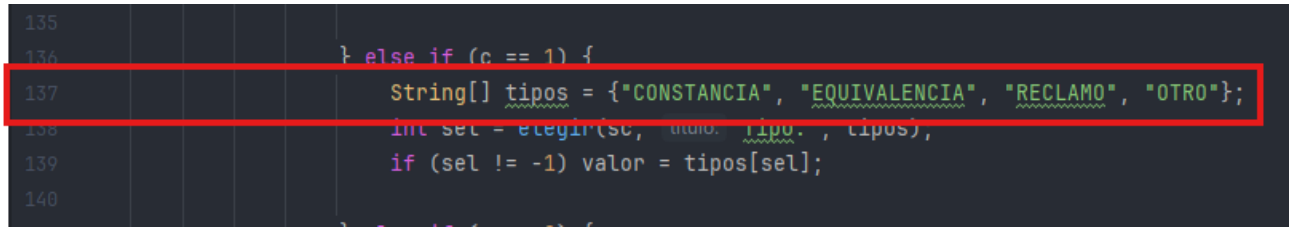
CAP-18

```
public class Solicitud {  23 usages  Matias H
    public int id;  12 usages
    public Estudiante est;  5 usages
    public Responsable resp;  12 usages
    public String tipo;        // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5 usages
    public int prioridad;      // 1 = alta, 2 = media, 3 = baja  8 usages
    public Estado estado;  23 usages
    public String desc;  1 usage
    public String fecha;  2 usages
```

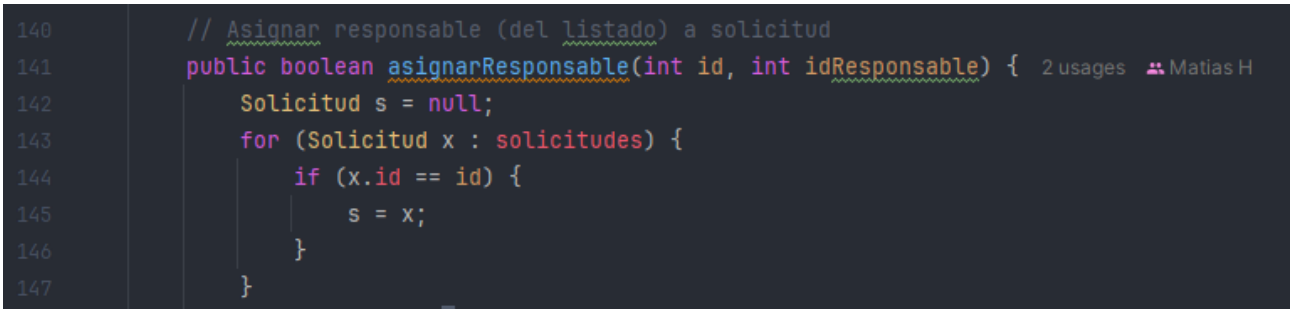
CAP-19

```
                String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
                int t = elegir(sc, titulo: "Tipo:", tipos);
                if (t == -1) {
                    System.out.println("Registro cancelado.");
```

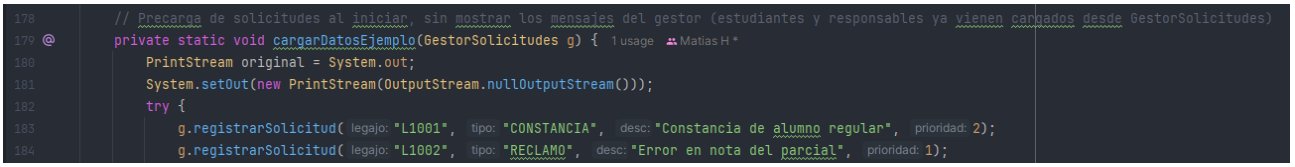
CAP-20



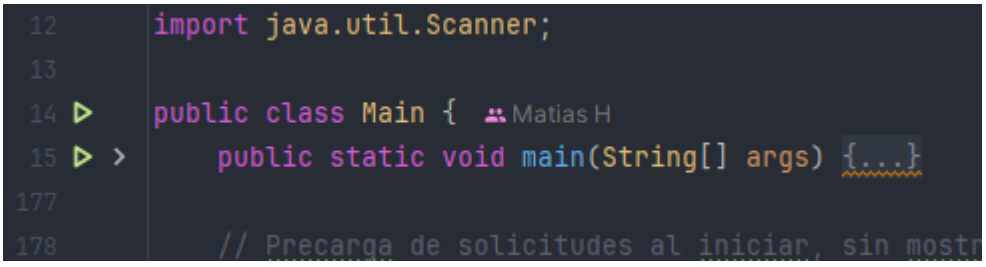
CAP-21



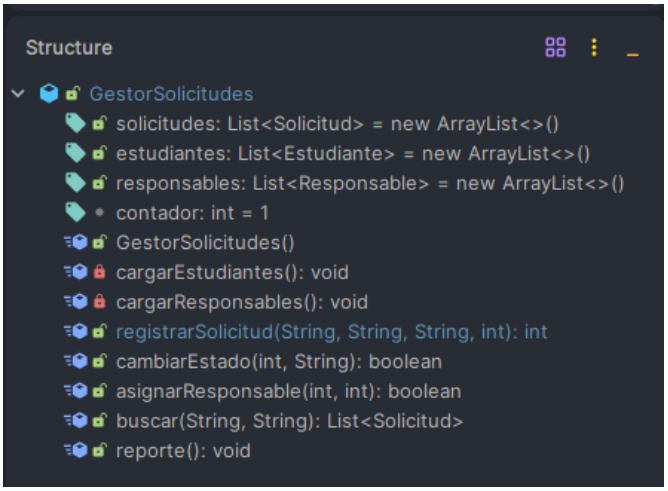
CAP-22



CAP-23



CAP-24


Structure
GestorSolicitudes
solicitudes: List<Solicitud> = new ArrayList<>()
estudiantes: List<Estudiante> = new ArrayList<>()
responsables: List<Responsable> = new ArrayList<>()
contador: int = 1
GestorSolicitudes()
cargarEstudiantes(): void
cargarResponsables(): void
registrarSolicitud(String, String, String, int): int
cambiarEstado(int, String): boolean
asignarResponsable(int, int): boolean
buscar(String, String): List<Solicitud>
reporte(): void

CAP-25


GestorSolicitudes.java
public class GestorSolicitudes { 6 usages Matias H
// Registrar solicitud
public int registrarSolicitud(String legajo, String tipo, String desc, int prioridad) { 4 usages Matias H
Estudiante e = null;
for (Estudiante x : estudiantes) {
if (x.legajo.equals(legajo)) {
e = x;
}
}
if (e == null) {
System.out.println("Error: estudiante no encontrado");
return -1;
}
if (!tipo.equals("CONSTANCIA") && !tipo.equals("EQUIVALENCIA")
&& !tipo.equals("RECLAMO") && !tipo.equals("OTRO")) {
System.out.println("Error: tipo invalido");
return -1;
}
if (prioridad < 1 || prioridad > 3) {
System.out.println("Error: prioridad invalida");
return -1;
}
if (desc == null || desc.length() < 5) {
System.out.println("Error: descripcion muy corta");
return -1;
}

CAP-26

```java
public int registrarSolicitud(String legajo, String tipo, String desc, int prioridad) {
    Estudiante e = null;
    for (Estudiante x : estudiantes) {
        if (x.legajo.equals(legajo)) {
            e = x;
        }
    }
    if (e == null) {
        System.out.println("Error: estudiante no encontrado");
        return -1;
    }
    if (!tipo.equals("CONSTANCIA") && !tipo.equals("EQUIVALENCIA")
            && !tipo.equals("RECLAMO") && !tipo.equals("OTRO")) {
        System.out.println("Error: tipo invalido");
        return -1;
    }
    if (prioridad < 1 || prioridad > 3) {
        System.out.println("Error: prioridad invalida");
        return -1;
    }
    if (desc == null || desc.length() < 5) {
        System.out.println("Error: descripcion muy corta");
        return -1;
    }
```

CAP-27

```java
// Cambiar estado de solicitud
public boolean cambiarEstado(int id, String nuevoEstado) {  2 usages  Matias H
    Solicitud s = null;
    for (Solicitud x : solicitudes) {
        if (x.id == id) {
            s = x;
        }
    }
    if (s == null) {
        System.out.println("Error: solicitud no encontrada");
        return false;
    }
    if (nuevoEstado.equals("EN_PROCESO")) {
        if (s.estado == Estado.PENDIENTE) {
            if (s.resp == null) {
                System.out.println("Error: debe asignar responsable primero");
                return false;
            }
            s.estado = Estado.EN_PROCESO;
        } else {
            System.out.println("Error: transicion invalida");
            return false;
        }
    } else if (nuevoEstado.equals("RESUELTA")) {
        if (s.estado == Estado.EN_PROCESO) {
            s.estado = Estado.RESUELTA;
        } else {
            System.out.println("Error: transicion invalida");
            return false;
        }
```

CAP-28

```
140     // Asignar responsable (del listado) a solicitud
141     public boolean asignarResponsable(int id, int idResponsable) {  2 usages
142         Solicitud s = null;
143         for (Solicitud x : solicitudes) {
144             if (x.id == id) {
145                 s = x;
146             }
147         }
148         if (s == null) {
149             System.out.println("Error: solicitud no encontrada");
150             return false;
151         }
152         if (s.estado == Estado.RESUELTA || s.estado == Estado.RECHAZADA) {
153             System.out.println("Error: la solicitud esta cerrada");
154             return false;
155         }
156         Responsable r = null;
157         for (Responsable x : responsables) {
158             if (x.id == idResponsable) {
```

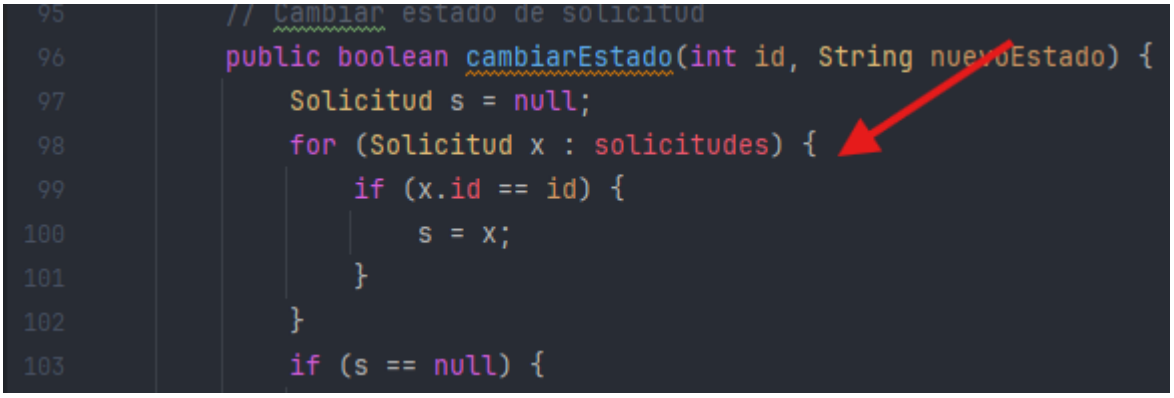
CAP-29

```
196                 }
197             }
198         } else {
199             System.out.println("Criterio no soportado");
200         }
201         return res;
```

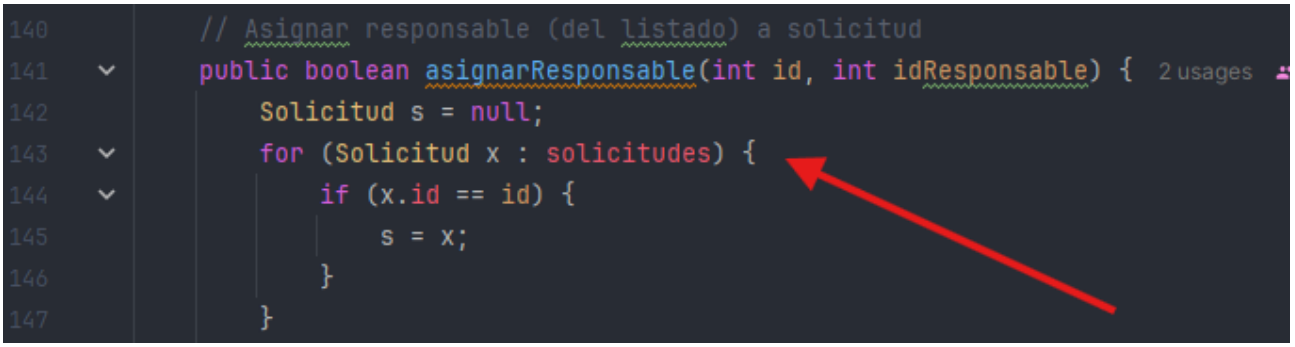
CAP-30

```
194 @   private static Solicitud pedirSolicitud(Scanner sc, GestorSolicitudes g)
195         System.out.print("Id solicitud: ");
196         try {
197             int id = Integer.parseInt(sc.nextLine().trim());
198             for (Solicitud x : g.solicitudes) {
199                 if (x.id == id) {
200                     return x;
201                 }
202             }
203         } catch (NumberFormatException ex) {
204             // cae al return null
205         }
206         return null;
207     }
```

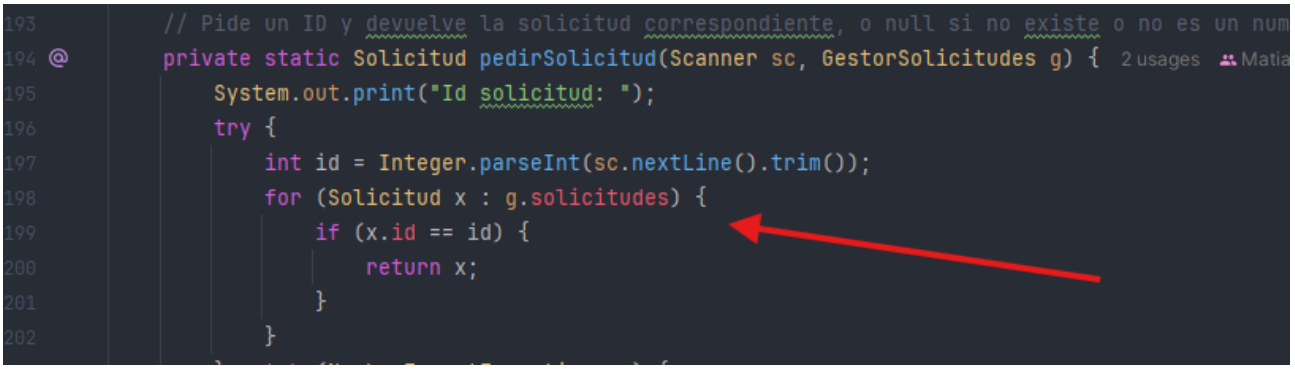
CAP-31



CAP-32



CAP-33



CAP-34

```
71                  if (sol == null) {
72                      System.out.println("Error: solicitud no encontrada");
73                  } else if (sol.estado == Estado.RESUELTA || sol.estado == Estado.RECHAZADA) {
74                      System.out.println("Error: la solicitud esta cerrada");
75                  } else {
76                      String[] estados;
77                      if (sol.resp == null) {
78                          System.out.println("La solicitud #" + sol.id + " no tiene responsable asignado. "
79                                  + "Solo puede rechazarse; para otros estados asignelo desde la opcion 3.");
80                          estados = new String[]{"RECHAZADA"};
81                      } else {
82                          estados = new String[]{"EN_PROCESO", "RESUELTA", "RECHAZADA"};
83                      }
84                      int e = elegir(sc, titulo: "Nuevo estado:", estados);
85                      if (e != -1) {
86                          g.cambiarEstado(sol.id, estados[e]);
87                      }
88                  }
89              }
90
91          } else if (op.equals("3")) {
92              if (g.solicitudes.isEmpty()) {
93                  System.out.println("No hay solicitudes cargadas.");
94              } else {
95                  listarSolicitudes(g);
96                  Solicitud sol = pedirSolicitud(sc, g);
97
98                  if (sol == null) {
99                      System.out.println("Error: solicitud no encontrada");
100                 } else if (sol.estado == Estado.RESUELTA || sol.estado == Estado.RECHAZADA) {
101                     System.out.println("Error: la solicitud esta cerrada");
102                 } else {
```

CAP-35

```
47                  } else {
48                      Estudiante est = g.estudiantes.get(sEst);
49
50                      String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
51                      int t = elegir(sc, titulo: "Tipo:", tipos);
52                      if (t == -1) {
53                          System.out.println("Registro cancelado.");
54                      } else {
55                          System.out.print("Descripcion: ");
```

CAP-36

```
135
136                 } else if (c == 1) {
137                     String[] tipos = {"CONSTANCIA", "EQUIVALENCIA", "RECLAMO", "OTRO"};
138                     int sel = elegir(sc, titulo: "Tipo:", tipos);
139                     if (sel != -1) valor = tipos[sel];
140
141                 } else if (c == 2) {
                        if (g.estudiantes.isEmpty()) {
```

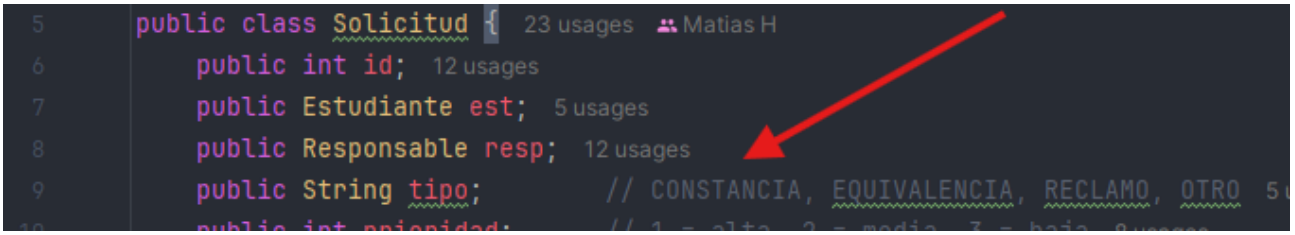
CAP-37

```
72                  return -1;
73              }
74              if (!tipo.equals("CONSTANCIA") && !tipo.equals("EQUIVALENCIA")
75                      && !tipo.equals("RECLAMO") && !tipo.equals("OTRO")) {
76                  System.out.println("Error: tipo invalido");
77                  return -1;
78              }
79              if (prioridad < 1 || prioridad > 3) {
80                  System.out.println("Error: prioridad invalida");
```

CAP-38

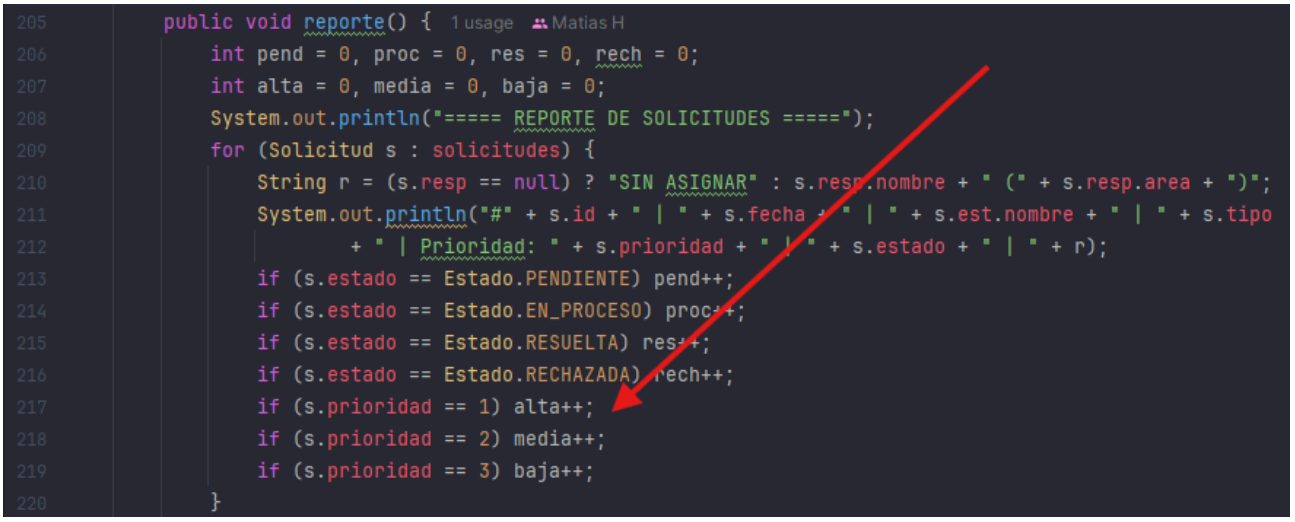
```
5   public class Solicitud {  23 usages  Matias H
6       public int id;  12 usages
7       public Estudiante est;  5 usages
8       public Responsable resp;  12 usages
9       public String tipo;         // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5
10      public int prioridad;       // 1 = alta, 2 = media, 3 = baja  8 usages
```

CAP-39



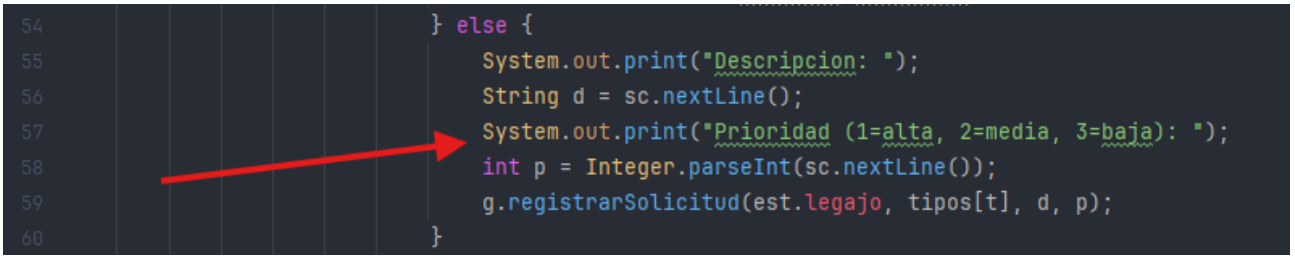
```
public class Solicitud {  23 usages  Matias H
    public int id;  12 usages
    public Estudiante est;  5 usages
    public Responsable resp;  12 usages
    public String tipo;          // CONSTANCIA, EQUIVALENCIA, RECLAMO, OTRO  5
```

CAP-40



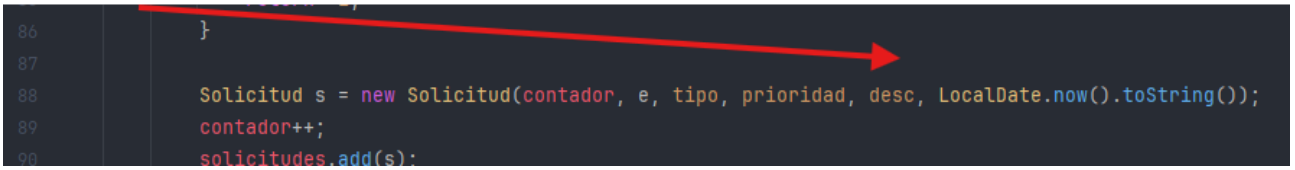
```
public void reporte() {  1 usage  Matias H
    int pend = 0, proc = 0, res = 0, rech = 0;
    int alta = 0, media = 0, baja = 0;
    System.out.println("===== REPORTE DE SOLICITUDES =====");
    for (Solicitud s : solicitudes) {
        String r = (s.resp == null) ? "SIN ASIGNAR" : s.resp.nombre + " (" + s.resp.area + ")";
        System.out.println("#" + s.id + " | " + s.fecha + " | " + s.est.nombre + " | " + s.tipo
                + " | Prioridad: " + s.prioridad + " | " + s.estado + " | " + r);
        if (s.estado == Estado.PENDIENTE) pend++;
        if (s.estado == Estado.EN_PROCESO) proc++;
        if (s.estado == Estado.RESUELTA) res++;
        if (s.estado == Estado.RECHAZADA) rech++;
        if (s.prioridad == 1) alta++;
        if (s.prioridad == 2) media++;
        if (s.prioridad == 3) baja++;
    }
```

CAP-41



```
        } else {
            System.out.print("Descripcion: ");
            String d = sc.nextLine();
            System.out.print("Prioridad (1=alta, 2=media, 3=baja): ");
            int p = Integer.parseInt(sc.nextLine());
            g.registrarSolicitud(est.legajo, tipos[t], d, p);
        }
```

CAP-42



```
    }

    Solicitud s = new Solicitud(contador, e, tipo, prioridad, desc, LocalDate.now().toString());
    contador++;
    solicitudes.add(s);
```

CAP-43

```
192         } else if (criterio.equals("prioridad")) {
193             for (Solicitud s : solicitudes) {
194                 if (String.valueOf(s.prioridad).equals(valor)) {
195                     res.add(s);
196                 }
197             }
```

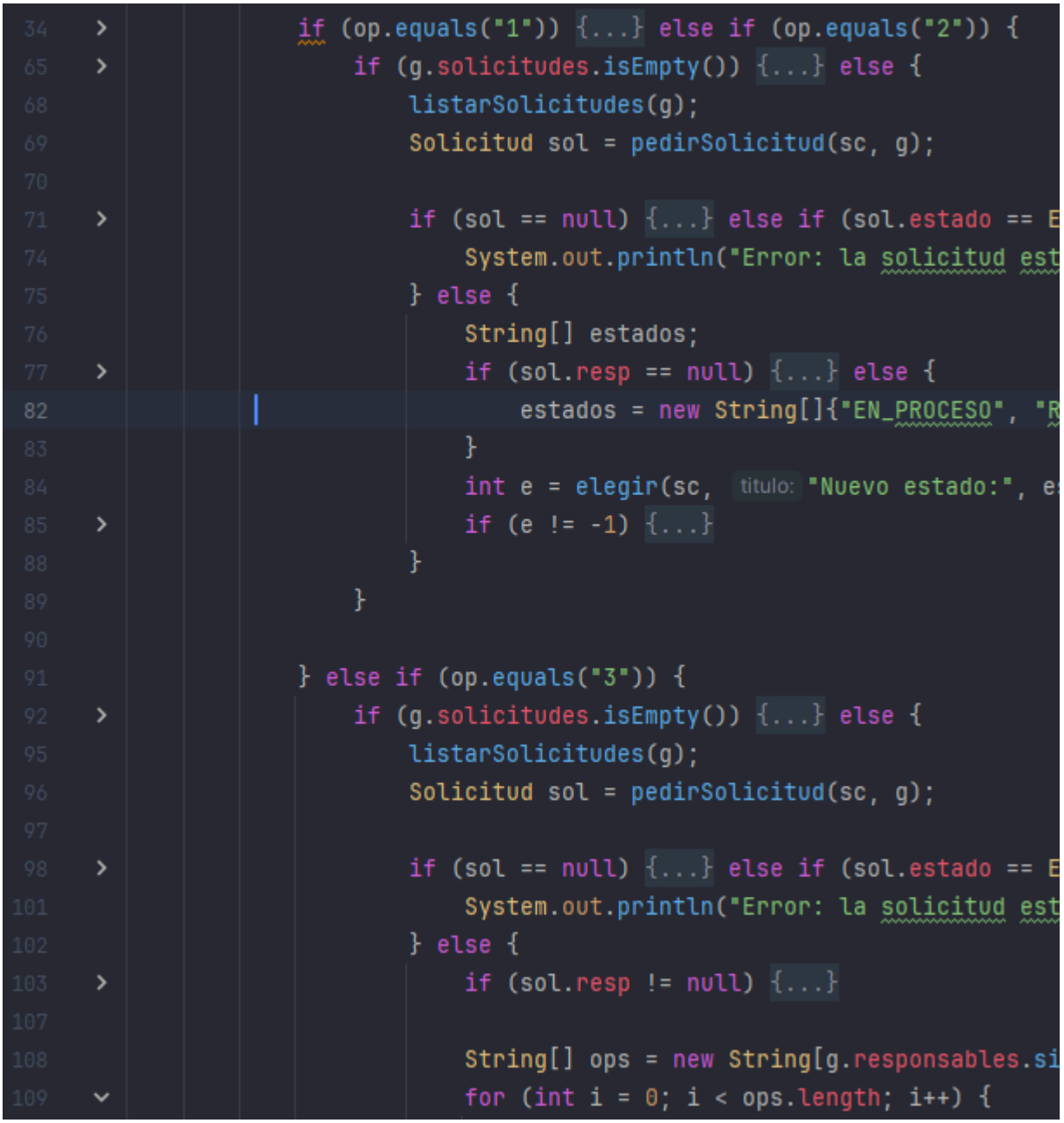
CAP-44

```
205     public void reporte() {  1 usage   Matias H
206         int pend = 0, proc = 0, res = 0, rech = 0;
207         int alta = 0, media = 0, baja = 0;
208         System.out.println("===== REPORTE DE SOLICITUDES ====
209         for (Solicitud s : solicitudes) {
210             String r = (s.resp == null) ? "SIN ASIGNAR" : s.
211             System.out.println("#" + s.id + " | " + s.fecha +
212                     + " | Prioridad: " + s.prioridad + " | "
213             if (s.estado == Estado.PENDIENTE) pend++;
214             if (s.estado == Estado.EN_PROCESO) proc++;
215             if (s.estado == Estado.RESUELTA) res++;
216             if (s.estado == Estado.RECHAZADA) rech++;
217             if (s.prioridad == 1) alta++;
218             if (s.prioridad == 2) media++;
219             if (s.prioridad == 3) baja++;
220         }
```

CAP-45

```java
    // Buscar / Filtrar
    public List<Solicitud> buscar(String criterio, String valor) {  1 usage
        List<Solicitud> res = new ArrayList<>();
        if (criterio.equals("estado")) {
            for (Solicitud s : solicitudes) {
                if (s.estado.toString().equalsIgnoreCase(valor)) {
                    res.add(s);
                }
            }
        } else if (criterio.equals("tipo")) {
            for (Solicitud s : solicitudes) {
                if (s.tipo.equalsIgnoreCase(valor)) {
                    res.add(s);
                }
            }
        } else if (criterio.equals("legajo")) {
            for (Solicitud s : solicitudes) {
                if (s.est.legajo.equals(valor)) {
                    res.add(s);
                }
            }
        } else if (criterio.equals("prioridad")) {
            for (Solicitud s : solicitudes) {
                if (String.valueOf(s.prioridad).equals(valor)) {
                    res.add(s);
                }
            }
        } else {
            System.out.println("Criterio no soportado");
        }
        return res;
```

CAP-46



```
34      if (op.equals("1")) {...} else if (op.equals("2")) {
65          if (g.solicitudes.isEmpty()) {...} else {
68              listarSolicitudes(g);
69              Solicitud sol = pedirSolicitud(sc, g);
70
71              if (sol == null) {...} else if (sol.estado == E
74                  System.out.println("Error: la solicitud est
75              } else {
76                  String[] estados;
77                  if (sol.resp == null) {...} else {
82                      estados = new String[]{"EN_PROCESO", "R
83                  }
84                  int e = elegir(sc, titulo: "Nuevo estado:", e
85                  if (e != -1) {...}
88              }
89          }
90
91      } else if (op.equals("3")) {
92          if (g.solicitudes.isEmpty()) {...} else {
95              listarSolicitudes(g);
96              Solicitud sol = pedirSolicitud(sc, g);
97
98              if (sol == null) {...} else if (sol.estado == E
101                 System.out.println("Error: la solicitud est
102             } else {
103                 if (sol.resp != null) {...}
107
108                 String[] ops = new String[g.responsables.si
109                 for (int i = 0; i < ops.length; i++) {
```

CAP-47

```
4
5   public class Solicitud {  23 usages  Matias
6       public int id;  12 usages
7       public Estudiante est;  5 usages
8       public Responsable resp;  12 usages
9       public String tipo;          // CONSTANC
10      public int prioridad;        // 1 = alta
11      public Estado estado;  23 usages
12      public String desc;  1 usage
13      public String fecha;  2 usages
```

CAP-48

```
171     // Buscar / Filtrar
172 @   public List<Solicitud> buscar(String criterio, String valor) {
173         List<Solicitud> res = new ArrayList<>();
```

CAP-49

```
64              } else if (op.equals("2")) {
65                  if (g.solicitudes.isEmpty()) {
66                      System.out.println("No hay so
```

CAP-50

```
107             if (nuevoEstado.equals("EN_PROCESO")) {
108                 if (s.estado == Estado.PENDIENTE) {
109                     if (s.resp == null) {
110                         System.out.println("Error: deb
111                         return false;
112                     }
113                     s.estado = Estado.EN_PROCESO;
114                 } else {
```

CAP-51

```
71              if (sol == null) {
72                  System.out.println("Error: solicitud no encontrada");
73              } else if (sol.estado == Estado.RESUELTA || sol.estado == Estado.RECHAZADA) {
74                  System.out.println("Error: la solicitud esta cerrada");
75              } else {
```

CAP-52

```
122             } else if (op.equals("4")) {
123                 String[] criterios = {"estado", "tipo", "legajo", "pr
124                 int c = elegir(sc, titulo: "Buscar por:", new String[]{
125                 String valor = null;
126
127                 if (c == 0) {
128                     Estado[] es = Estado.values();
129                     String[] nombres = new String[es.length];
130                     for (int i = 0; i < es.length; i++) {
131                         nombres[i] = es[i].toString();
132                     }
133                     int sel = elegir(sc, titulo: "Estado:", nombres);
134                     if (sel != -1) valor = nombres[sel];
135
```

CAP-53

```
9   public class GestorSolicitudes {  6 usages  Matias H
10
11      public List<Solicitud> solicitudes = new ArrayList<>();
12      public List<Estudiante> estudiantes = new ArrayList<>();
13      public List<Responsable> responsables = new ArrayList<>();
```

CAP-54

```
178     // Precarga de solicitudes al iniciar, sin mostrar los mensajes del gestor (estudiantes y responsables ya vienen ca
179 @   private static void cargarDatosEjemplo(GestorSolicitudes g) {  1 usage  Matias H
180         PrintStream original = System.out;
181         System.setOut(new PrintStream(OutputStream.nullOutputStream()));
182         try {
183             g.registrarSolicitud( legajo: "L1001",  tipo: "CONSTANCIA",  desc: "Constancia de alumno regular",  prioridad: 2);
184             g.registrarSolicitud( legajo: "L1002",  tipo: "RECLAMO",  desc: "Error en nota del parcial",  prioridad: 1);
185             g.registrarSolicitud( legajo: "L1001",  tipo: "EQUIVALENCIA",  desc: "Equivalencia de Matematica I",  prioridad: 3);
186             g.asignarResponsable( id: 2,  idResponsable: 1);    // Maria Lopez (Alumnado)
187             g.cambiarEstado( id: 2,  nuevoEstado: "EN_PROCESO");
188         } finally {
189             System.setOut(original);
190         }
191     }
```

CAP-55

```
15      public Solicitud(int id, Estudiante est, String tipo, int prioridad, String desc, String fecha)
16          this.id = id;
17          this.est = est;
18          this.tipo = tipo;
19          this.prioridad = prioridad;
20          this.desc = desc;
21          this.fecha = fecha;
22          this.estado = Estado.PENDIENTE;
23      }
```

CAP-56

Etapa 3: Deuda técnica y code smells

3.1 Criterio de diagnóstico

Se revisó el código de `estado-inicial` buscando los code smells del catálogo de Fowler y debilidades técnicas generales. Cada hallazgo se registra con:

- **Evidencia:** qué se observa en el código.
- **Ubicación:** archivo y líneas.
- **Impacto:** qué consecuencia tiene hoy sobre la legibilidad, la mantenibilidad o la evolución.
- **Riesgo de mantenimiento:** qué puede salir mal al modificar el código.
- **Prioridad de corrección:**
  - **Alta:** causa defectos visibles, bloquea otras mejoras (por ejemplo las pruebas) o concentra la mayor parte de los cambios futuros.
  - **Media:** dificulta el mantenimiento pero no rompe nada por sí solo.
  - **Baja:** afecta la prolijidad y su corrección es barata y sin riesgo.

La deuda de la versión inicial es **deliberada y prudente**: el sistema se construyó sin criterios de diseño a propósito, para tener un punto de partida diagnosticable. Su "interés" es el costo extra que tiene cada cambio mientras la deuda siga ahí.

3.2 Resumen de hallazgos

ID	Code smell / debilidad	Ubicación principal	Prioridad
CS-01	Long Method	`Main.main()` (15–176)	Alta
CS-02	Large Class / God Class	`GestorSolicitudes`	Alta
CS-03	Lógica de negocio acoplada a la consola	`GestorSolicitudes` (23 `System.out`), `Main.cargarDatosEjemplo()`	Alta
CS-04	Manejo de errores inconsistente	`GestorSolicitudes`, `Main` (58, 203–205, 221–223)	Alta
CS-05	Duplicated Code	`Main` y `GestorSolicitudes` (varios)	Alta
CS-06	Shotgun Surgery	Tipos y estados dispersos en 4 archivos	Alta

CAP-57

es realmente necesario señalar todas las lineas de código en todos los casos dentro de cada code smell?

No. La consigna pide **ubicación** para cada smell, no un inventario de todas las líneas. Con archivo, método y un rango de líneas, o la línea más representativa, alcanza para que el docente lo encuentre. Listar todo agrega ruido y hace más fácil equivocarse con un número de línea.

Las líneas exactas solo valen la pena cuando la cantidad de lugares es justamente la evidencia:

- **CS-05 Duplicated Code:** hay que mostrar que la misma lógica aparece en varios sitios. Ahí sí conviene la tabla con las ubicaciones, aunque pueden quedarse con 3 o 4 filas en vez de 6.
- **CS-06 Shotgun Surgery:** el punto es cuántos lugares hay que tocar para un cambio chico. Igual pueden resumirlo así: "agregar un tipo obliga a modificar 4 lugares en 3 archivos (`Main`, `GestorSolicitudes`, `Solicitud`)", sin detallar cada línea. La captura de la búsqueda ya muestra el resto.

Para los demás, una ubicación resumida es suficiente:

Smell	Ubicación suficiente
CS-01	`Main.main()`, líneas 15–176
CS-02	`GestorSolicitudes` (clase completa)
CS-03	`GestorSolicitudes` (23 `System.out`, todos sus métodos) y `Main.cargarDatosEjemplo()`
CS-04	`Main`, línea 58; métodos públicos de `GestorSolicitudes`
CS-07	`Solicitud`: campos `tipo`, `prioridad`, `fecha`
CS-08	`GestorSolicitudes.buscar()` y `cambiarEstado()`

2. Uso de IA 🤖

2.1. Declaración general

Campo	Detalle
Herramientas utilizadas	Claude (Anthropic), ChatGPT (OpenAI)
Finalidad general	Comprensión de conceptos, revisión de alternativas, mejora de redacción

2.2 Escala de grado de intervención

Nivel	Descripción
Bajo	Consulta conceptual o aclaración de dudas. No se incorporó texto ni código generado.
Medio	Se pidieron alternativas o sugerencias. El equipo las analizó y adaptó; la decisión final fue propia.
Alto	Se incorporó contenido generado (texto o código) con modificaciones menores, revisado y validado por el equipo.

2.3 Tabla resumen de interacciones

ID	Etapa	IA	Finalidad	Grado	Evidencia
IA-01	1 - Selección y descripción del sistema base	Claude	Borrador del código para la versión inicial del sistema	Medio	CA-01
IA-02	1 - Selección y descripción del sistema base	Claude	Template para README	Medio	CA-02
IA-03	2 - Calidad	ChatGPT	Infograma para el documento	Alto	CA-03
IA-04	2 - Calidad	Claude	Métricas UX	Alto	CA-04
IA-05					
IA-06					
IA-07					
IA-08					

2.4 Registro por interacción

IA-01

Campo	Detalle
Etapa	1 - Selección y descripción del sistema base
Herramienta	Claude
Objetivo de la consulta	Borrador del código para la versión inicial del sistema
Prompt	En relación con la consigna, el proyecto Java ya se encuentra creado en blanco y configurado con Gradle. Preciso que te enfoques en los puntos 3 y 4 de la consigna y que generes de manera individual una versión inicial de los archivos para el proyecto, incluyendo las entidades mínimas y los casos de uso mínimos allí especificados. Prioriza una implementación inicial y funcional, sin buscar una solución perfecta ni realizar refactorizaciones anticipadas, dado que posteriormente se deberán desarrollar los puntos restantes de la consigna.
Uso dado a la respuesta	Se tomó la estructura base propuesta por la IA: las entidades (Solicitud, Estudiante, Responsable y Estado), los cinco casos de uso (registrar, cambiar estado, asignar responsable, buscar/filtrar y reporte) y el menú de consola, que se adaptaron al paquete propio tupad.ms2.model. Se descartó la persistencia en archivo CSV (guardar() y cargar()) y se optó por memoria estructurada, que el enunciado admite y que simplifica el sistema base.
Grado de intervención	Medio
Evidencia	CAP-01

IA-02

Campo	Detalle
Etapa	1 - Selección y descripción del sistema base
Herramienta	Claude
Objetivo de la consulta	Template para README inicial
Prompt	En relación a la etapa 1 de la consigna, dame un borrador del README para utilizar en la entrega 1
Uso dado a la respuesta	Se tomó la estructura base propuesta por la IA: las entidades (Solicitud, Estudiante, Responsable y Estado), los cinco casos de uso (registrar, cambiar estado, asignar responsable, buscar/filtrar y reporte) y el menú de consola, que se adaptaron al paquete propio tupad.ms2.model. Se descartó la persistencia en archivo CSV (guardar() y cargar()) y se optó por memoria estructurada, que el enunciado admite y que simplifica el sistema base.
Grado de intervención	Medio
Evidencia	CAP-02

IA-03

Campo	Detalle
Etapa	2 - Calidad
Herramienta	ChatGPT
Objetivo de la consulta	Crear un infograma sobre las calidades, para el documento
Prompt	Crea un infograma minimalista, tipo banner, muy resumido, sobre este párrafo:: [texto en sección “Sobre la calidad” en el documento]
Uso dado a la respuesta	infograma para el documento
Grado de intervención	Medio
Evidencia	CAP-03

IA-04

Campo	Detalle
Etapa	2 - Calidad
Herramienta	Claude
Objetivo de la consulta	Métricas de UX para calidad externa
Prompt	Dame un template de metricas de UX para el informe de calidad externa, con datos ficticios pero que reflejen la calidad actual del software.
Uso dado a la respuesta	Tablas de métricas
Grado de intervención	Alto
Evidencia	CAP-04

IA-05

Campo	Detalle
Etapa	3 - Code Smells
Herramienta	Claude
Objetivo de la consulta	Listar code smells
Prompt	En la Etapa 3 tenemos que identificar al menos 8 code smells o debilidades técnicas en nuestro sistema base (gestión de solicitudes académicas, en Java, con consola y persistencia en memoria). Te adjunto el código de la versión estado-inicial. Necesitamos solo una lista de los code smells que detectes, con este formato por cada uno: -Nombre del smell (según el catálogo de Fowler, si corresponde) -Archivo y método donde aparece -Una frase breve que indique qué lo evidencia
Uso dado a la respuesta	Resumen y listado
Grado de intervención	Alto
Evidencia	CAP-56

IA-06

Campo	Detalle
Etapa	3 - Code Smells
Herramienta	Claude
Objetivo de la consulta	Simplificación de las ubicaciones en el informe
Prompt	Es realmente necesario señalar todas las líneas de código en todos los casos dentro de cada code smell?
Uso dado a la respuesta	Aclaración
Grado de intervención	Alto
Evidencia	CAP-56

3. Bibliografía y recursos consultados

- Code Smells https://refactoring.guru/refactoring/smells